Python fundamentals

Data Types

- Data types define the kind of value stored in a variable.
- Python automatically determines the data type based on the assigned value.
- Common data types are int, float, string, and boolean.
- Integers represent whole numbers such as 10 and 25.
- Floats represent decimal numbers such as 10.5 and 3.14.
- Strings store text values enclosed within quotes.

Variables

- Variables are used to store data in memory.
- Values are assigned using the = operator.
- Variable names should be meaningful and descriptive.
- Variables can store different data types.
- Stored values can be modified during program execution.
- Variables make calculations and data manipulation easier.

Input and Output

- input() is used to collect information from the user.
- User input is returned as a string by default.
- print() is used to display output on the screen.
- Input and output make programs interactive.
- Messages can be customized for better user experience.
- Most programs rely on input and output operations.

Type Conversion

- Type conversion changes one data type into another.
- int() converts values into integers.
- float() converts values into decimal numbers.
- str() converts values into strings.
- Conversion is commonly used with user input.
- Incorrect conversions may raise errors.

Conditional Statements

- Conditional statements help programs make decisions.
- Conditions evaluate to either True or False.
- They allow different actions based on different situations.
- Decision-making makes programs more dynamic.
- Conditions are commonly used with comparison operators.
- Indentation defines conditional code blocks.

If Statement

- if executes code when a condition is True.
- It is the simplest form of decision-making.
- Conditions are checked before execution.
- False conditions skip the code block.
- Multiple if statements can exist in a program.
- It helps control program flow.

If-Elif-Else Statement

- elif checks additional conditions.
- else executes when all previous conditions are False.
- Only one matching block executes.
- Multiple alternatives can be handled efficiently.
- It reduces the need for multiple separate if statements.
- It improves readability and logic.

Comparison Operators

- == checks if two values are equal.
- != checks if two values are different.
- > and < compare greater-than and less-than relationships.
- >= and <= include equality in comparisons.
- Comparisons return Boolean values.
- They are widely used in conditional statements.

Logical Operators

- and requires all conditions to be True.
- or requires at least one condition to be True.
- not reverses a Boolean value.
- Logical operators combine multiple conditions.
- They create more powerful decision-making rules.
- They improve flexibility in programs.

Boolean Data Type

- Boolean values are either True or False.
- They represent logical outcomes.
- Comparisons produce Boolean results.
- Booleans are heavily used in conditions.
- They help control program execution.
- Boolean variables improve code readability.

Lists

- Lists store multiple values in a single variable.
- Elements are stored in an ordered sequence.
- Lists can contain different data types.
- Indexing starts from 0 in Python.
- Lists are mutable and can be modified.
- They are commonly used for storing datasets.

List Indexing

- Every list item has an index position.
- The first element is at index 0.
- Elements are accessed using square brackets.
- Indexes allow direct access to data.
- Incorrect indexes raise errors.
- Indexing makes data retrieval efficient.

List Methods

- append() adds new elements to a list.
- remove() deletes elements by value.
- del removes elements by index.
- Methods modify the existing list.
- Lists can grow dynamically.
- Built-in methods simplify list operations.

Loops

- Loops repeat a block of code multiple times.
- They reduce repetitive coding.
- Loops improve efficiency and automation.
- Python mainly uses for and while loops.
- Loop execution depends on conditions or collections.
- Loops are essential for data processing.

For Loops

- For loops iterate through collections.
- They process one element at a time.
- A temporary variable stores the current item.
- Lists are commonly traversed using for loops.
- Indentation defines the loop body.
- They simplify repetitive operations.

range() Function

- range() generates a sequence of numbers.
- It is frequently used with for loops.
- Start, stop, and step values can be specified.
- The stop value is excluded from the sequence.
- It helps repeat tasks a fixed number of times.
- Range improves loop control.

Dictionaries

- Dictionaries store data as key-value pairs.
- Keys are used to retrieve associated values.
- Keys must be unique.
- Dictionaries provide fast data lookup.
- Values can be of any data type.
- They are widely used in real-world applications.

Dictionary Operations

- Values are accessed using their keys.
- New key-value pairs can be added dynamically.
- Existing values can be updated.
- Items can be removed using del.
- get() safely retrieves values.
- Dictionary operations are efficient and flexible.

Dictionary Methods

- get() returns a value for a given key.
- items() returns key-value pairs.
- keys() returns all dictionary keys.
- values() returns all stored values.
- Methods simplify dictionary manipulation.
- They are useful in data analysis tasks.

Sets

- Sets store unique values only.
- Duplicate elements are automatically removed.
- Sets are unordered collections.
- They support mathematical set operations.
- Sets are useful for removing duplicates.
- Membership testing is very fast.

Set Operations

- add() inserts new elements.
- remove() deletes elements.
- Union combines elements from multiple sets.
- Intersection finds common elements.
- Difference finds unique elements.
- Set operations are widely used in data cleaning.

Classes and Objects

- Classes act as blueprints for creating objects.
- Objects are instances of classes.
- Objects contain attributes and methods.
- OOP organizes programs around data and behavior.
- Classes improve code structure and reusability.
- They help model real-world entities.

Object State and Behavior

- State represents an object's data.
- Behavior represents actions an object performs.
- Attributes store state information.
- Methods define behavior.
- Different objects can have different states.
- Objects combine data and functionality.

Constructor (init)

- __init__() initializes object attributes.
- It runs automatically when an object is created.
- Constructor parameters receive initial values.
- self refers to the current object.
- Constructors simplify object setup.
- They ensure consistent object creation.

Methods

- Methods are functions defined inside classes.
- The first parameter is always self.
- Methods access object attributes.
- They define object behavior.
- Methods are called using dot notation.
- They improve code organization.

Inheritance

- Inheritance allows a class to reuse another class's features.
- Parent classes provide common functionality.
- Child classes inherit attributes and methods.
- It reduces code duplication.
- Inheritance models real-world relationships.
- It improves maintainability.

Method Overriding

- Child classes can redefine inherited methods.
- Overriding customizes behavior.
- The method name remains the same.
- It supports polymorphism.
- Different objects can respond differently.
- It increases flexibility in design.

Exceptions

- Exceptions are runtime errors that interrupt execution.
- They occur even when syntax is correct.
- Common exceptions include KeyError and ZeroDivisionError.
- Unhandled exceptions may crash programs.
- Exception handling improves reliability.
- Python provides tools to manage errors gracefully.

Try-Except Block

- try contains code that may generate errors.
- except handles exceptions when they occur.
- Programs continue running after handling errors.
- Multiple exceptions can be handled separately.
- Error messages can be customized.
- It improves user experience and robustness.

Finally Block

- finally executes regardless of errors.
- It is commonly used for cleanup operations.
- Files and resources can be safely closed.
- It runs after try and except blocks.
- It ensures important code always executes.
- It improves program stability.

Raising Exceptions

- Exceptions can be generated manually using raise.
- Custom error messages can be provided.
- Raising exceptions helps validate input.
- It prevents invalid program states.
- Custom exceptions improve debugging.
- They make programs more reliable.

This order follows the natural learning path: